Information Processing Letters 108 (2008) 320–325

Contents lists available at ScienceDirect

Information Processing Letters

www.elsevier.com/locate/ipl

Optimal prefix and suffix queries on texts ☆

Maxime Crochemore [a,b], Costas S. Iliopoulos [a,1], M. Sohel Rahman [a,*,2,3]

[a] Algorithm Design Group, Department of Computer Science, [4] King's College London, Strand, London WC2R 2LS, England
[b] Institut Gaspard-Monge, University of Marne-la-Vallée, France

ARTICLE INFO

Article history:
Received 2 May 2007
Received in revised form 13 January 2008
Accepted 6 May 2008
Available online 20 June 2008
Communicated by A. Moffat

Keywords:
Algorithms
Combinatorial problems
Pattern matching
Data structures

ABSTRACT

In this paper, we study a restricted version of the position restricted pattern matching problem introduced and studied by Mäkinen and Navarro [V. Mäkinen, G. Navarro, Position-restricted substring searching, in: J.R. Correa, A. Hevia, M.A. Kiwi (Eds.), LATIN, in: Lecture Notes in Computer Science, vol. 3887, Springer, 2006, pp. 703–714]. In the problem handled in this paper, we are interested in those occurrences of the pattern that lies in a suffix or in a prefix of the given text. We achieve optimal query time for our problem against a data structure which is an extension of the classic suffix tree data structure. The time and space complexity of the data structure is dominated by that of the suffix tree. Notably, the (best) algorithm by Mäkinen and Navarro, if applied to our problem, gives sub-optimal query time and the corresponding data structure also requires more time and space.

1. Introduction

The classical pattern matching problem is to find all the occurrences of a given pattern $\mathcal{P} = \mathcal{P}[1..m]$ of length m in a text $\mathcal{T} = \mathcal{T}[1..n]$ of length n, both being sequences of characters drawn from a finite character set Σ. This problem, along with its numerous variants, has been the focus of extensive research in the field of computer science. The indexing problem for pattern matching, indexed pattern matching for short, is to preprocess a given text $\mathcal{T}[1..n]$ over an alphabet Σ as efficiently as possible to build a data structure to support the following form of online queries: Given a pattern $\mathcal{P}[1..m]$ over Σ find the occurrences of $\mathcal{P}$ in $\mathcal{T}$. The indexed pattern matching problem and its many variants have been central in pattern matching literature [8,20,13,18,19,15,11,1,5,26,27,32,34,33,6]. Recently, Mäkinen and Navarro, in [28], considered an interesting variant of indexed pattern matching, where only the occurrences of a given pattern starting in a particular area, are of interest. In particular, in this variant, the query provides an interval $[\ell..r], 1 \leqslant \ell \leqslant r \leqslant n$, along with the pattern $\mathcal{P}$ and the occurrences of $\mathcal{P}$ in $\mathcal{T}[\ell..r]$ are sought for. These queries, as is pointed out in [28], are fundamental in many text search situations where one wants to search only a part of the text. The authors in [28] presented a number of algorithms depending on different trade-offs between the time and space complexities. The best query time they achieved was $O(m + \log\log n + K)$ (K is the output size) against a data structure exhibiting $O(n \log^{1+\varepsilon} n)$ space and time complexity, where $0 < \varepsilon < 1$.

In this paper, we study a restricted version of the problem handled in [28]. In particular, we are interested in those occurrences of the pattern that lies in a suffix or in a prefix of the given text. In other words, in our case, the query interval $[\ell..r]$ is of special form: either $\ell = 1$, i.e. prefix search, or $r = n$, i.e. suffix search. This kind of queries seem to be interesting in many contexts as well. For example, many of the queries in real life are restricted up to the table of contents of a book or in the title and abstract of a

☆ Preliminary version appeared in [M. Crochemore, C.S. Iliopoulos, M.S. Rahman, Optimal prefix and suffix queries on texts, in: AofA, 2007, pp. 645–656].

* Corresponding author.
E-mail addresses: Maxime.Crochemore@kcl.ac.uk (M. Crochemore), csi@dcs.kcl.ac.uk (C.S. Iliopoulos), sohel@dcs.kcl.ac.uk (M.S. Rahman).

[1] Supported by EPSRC and Royal Society grants.
[2] Supported by the Commonwealth Scholarship Commission in the UK under the Commonwealth Scholarship and Fellowship Plan (CSFP).

[3] On Leave from Department of CSE, BUET, Dhaka-1000, Bangladesh.
[4] http://www.dcs.kcl.ac.uk/adg.

0020-0190/$ – see front matter
doi:10.1016/j.ipl.2008.05.027

scientific document. Our problem has application in a variant of the related and well-studied problem of *suffix–prefix matching* [17,23]. For a pair of strings (S_1, S_2), the suffix–prefix match of (S_1, S_2) is defined to be the longest suffix of string S_1 that matches a prefix of S_2. This problem can be solved efficiently using the solution to our problem as is detailed in Section 4.

This problem also gets motivation from bioinformatics, from sequencing and mapping of DNA [22,25] to be specific. Here, long strings of DNA must first be cut into smaller strings since only relatively small strings can be sequenced as a single piece. Now, known methods of doing that result in the pieces being randomly permuted. Hence, after obtaining and sequencing or mapping each of the smaller pieces, we need to determine the correct order of the small pieces. To solve this problem, we first make many copies of the DNA, and then cut up each copy with a different cutter so that pieces obtained from one cutter overlap pieces obtained from another. There exists numerous methods to do that. Then, each piece is sequenced or mapped, and in the case of mapping a string is created to indicate the order and type of the mapped features, where an alphabet has been created to represent the features. Given the entire set of sequenced or mapped pieces, the problem of assembling the proper DNA string can be modeled as the *shortest common superstring* problem: *find the shortest string which contains each of the other strings as a contiguous substring*. One of the approaches for solving this problem is the greedy approach [4,37,36]. In the greedy strategy for sequence assembly, this is usually done by finding markers close to the ends, i.e. suffixes of the strings these markers witness possible overlaps between a suffix of a sequence and a prefix of another sequence. Sequence having large overlaps are assembled in a longer sequence and so on. Clearly, in this strategy, one needs to look for patterns in the prefixes and suffixes of sequences.

In this paper, we present an efficient data structure to handle such online queries in the prefix or suffix of a given text in optimal time. Note that, the best query time achieved in [28] (for the more general problem) is not optimal due to the additional (mild) $\log\log n$ term. As a result, if applied to our problem, their algorithm exhibits sub-optimal query time and the corresponding data structure also requires more time and space. We further note that, very recently, optimal query time was achieved in [8,7] for the general problem handled in [28], albeit, against rather costly data structures. The bottleneck in the construction time of the data structure in [8,7] is due to an intermediate problem named the Range Next Value (RNV) problem. In [8], the RNV problem was solved using a data structure having $O(n^2)$ time and space complexity, which was later improved to $O(n^{1.5})$ in [7]. In both the cases, the space and time complexity of the main data structure are dominated by that of the data structure for RNV.

The rest of the paper is organized as follows. In Section 2, we present the preliminary concepts. The main result of this paper is presented in Section 3. In Section 4, we discuss some additional interesting problems that can be solved using our data structure. We conclude briefly in Section 5.

2. Preliminaries

A *text*, also called a *string*, is a sequence of zero or more symbols from an alphabet Σ. A text $\mathcal{T}$ of length n is denoted by $\mathcal{T}[1..n] = \mathcal{T}_1\mathcal{T}_2\ldots\mathcal{T}_n$, where $\mathcal{T}_i \in \Sigma$ for $1 \leqslant i \leqslant n$. The *length* of $\mathcal{T}$ is denoted by $|\mathcal{T}| = n$. A string w is a *factor* or *substring* of $\mathcal{T}$ if $\mathcal{T} = uwv$ for $u, v \in \Sigma^*$; in this case, the string w occurs at position $|u| + 1$ in $\mathcal{T}$. The factor w is denoted by $\mathcal{T}[|u| + 1..|u| + |w|]$. A *prefix* (*suffix*) of $\mathcal{T}$ is a factor $\mathcal{T}[x..y]$ such that $x = 1$ ($y = n$), $1 \leqslant y \leqslant n$ ($1 \leqslant x \leqslant n$). We define ith prefix to be the prefix ending at position i, i.e. $\mathcal{T}[1..i], 1 \leqslant i \leqslant n$. On the other hand, ith suffix is the suffix starting at position i, i.e. $\mathcal{T}[i..n], 1 \leqslant i \leqslant n$.

In traditional pattern matching problem, we want to find the occurrences of a given pattern $\mathcal{P}[1..m]$ in a text $\mathcal{T}[1..n]$. The pattern $\mathcal{P}$ is said to occur at position $i \in [1..n]$ of $\mathcal{T}$ if and only if $\mathcal{P} = \mathcal{T}[i..i+m-1]$. We use $Occ_{\mathcal{T}}^{\mathcal{P}}$ to denote the set of occurrences of $\mathcal{P}$ in $\mathcal{T}$.

The problem we handled in this paper can be defined formally as follows.

Problem "PMP/S" *(Pattern matching in a prefix/suffix).* We are given a text $\mathcal{T}$ of length n. Preprocess $\mathcal{T}$ to answer the following form of queries.

Query: Given a pattern $\mathcal{P}$ and a query interval $[\ell..r]$, with $1 \leqslant \ell \leqslant r \leqslant n$, where either $\ell = 1$ (prefix query) or $r = n$ (suffix query), construct the set

$$Occ_{\mathcal{T}[\ell..r]}^{\mathcal{P}} = \left\{ i \mid i \in Occ_{\mathcal{T}}^{\mathcal{P}} \text{ and } i \in [\ell..r] \right\}.$$

It is easy to realize that Problem PMP/S is a special case of the problem handled in [28]. Apart from being interesting from pure combinatorial point of view, Problem PMP/S is motivated by practical applications as discussed in Section 1. As a result, it is interesting to see whether the solution of [28] can be improved to optimal for this special case.

In traditional indexing problem one of the basic data structures used is the suffix tree data structure. In our indexing problem, we make use of this suffix tree data structure. A complete description of a suffix tree is beyond the scope of this paper, and can be found in [29,38] or in any textbook on stringology (e.g., [9,16]). However, for the sake of completeness, we define the suffix tree data structure as follows. Given a string $\mathcal{T}$ of length n over an alphabet Σ, the suffix tree $ST_{\mathcal{T}}$ of $\mathcal{T}$ is the compacted trie of all suffixes of $\mathcal{T}\$$, where $\$ \notin \Sigma$. Each leaf in $ST_{\mathcal{T}}$ represents a suffix $\mathcal{T}[i..n]$ of $\mathcal{T}$ and is labeled with the index i. We refer to the list (in left-to-right order) of indices of the leaves of the subtree rooted at node v as the leaf-list of v; it is denoted by $LL(v)$. Each edge in $ST_{\mathcal{T}}$ is labeled with a nonempty substring of $\mathcal{T}$ such that the path from the root to the leaf labeled with index i spells the suffix $\mathcal{T}[i..n]$. For any node v, we let ℓ_v denote the string obtained by concatenating the substrings labeling the edges on the path from the root to v in the order they appear. Several algorithms exist that can construct the suffix tree $ST_{\mathcal{T}}$ requiring linear space in $O(n \log \sigma)$ time, where $\sigma = \min(n, |\Sigma|)$ [29,38]. Notably, this means that, for bounded alphabet

the construction time is $O(n)$. Furthermore, in [10], Farach presented an $O(n)$ time algorithm for suffix tree construction, which works even for larger alphabets. In particular, Farach's algorithm works in linear time even if we have $\Sigma = \{1, \ldots, n^c\}$, where c is a constant. Given the suffix tree $ST_{\mathcal{T}}$ of a text $\mathcal{T}$ we define the "locus" $\mu^{\mathcal{P}}$ of a pattern $\mathcal{P}$ as the node in $ST_{\mathcal{T}}$ such that $\ell_{\mu^{\mathcal{P}}}$ has the prefix $\mathcal{P}$ and $|\ell_{\mu^{\mathcal{P}}}|$ is the smallest of all such nodes. Note that the locus of $\mathcal{P}$ does not exist, if $\mathcal{P}$ is not a substring of $\mathcal{T}$. Therefore, given $\mathcal{P}$, finding $\mu^{\mathcal{P}}$ suffices to determine whether $\mathcal{P}$ occurs in $\mathcal{T}$. Given a suffix tree of a text $\mathcal{T}$ and a pattern $\mathcal{P}$, one can find its locus and hence the fact whether $\mathcal{T}$ has an occurrence of $\mathcal{P}$ in $O(|\mathcal{P}| \log |\Sigma|)$ time. In addition to that, all such occurrences can be reported in constant time per occurrence. Finally, we note that, optimal query time of $O(|\mathcal{P}| + Occ_{\mathcal{T}}^{\mathcal{P}})$ can be achieved with suffix tree with $O(n \log n)$ bits of space [2,28].

3. An index for problem PMP/S

In this section, we handle Problem PMP/S. Our basic idea is to build an index data structure that would solve the problem in two steps. At first, it will (implicitly) give us the set $Occ_{\mathcal{T}}^{\mathcal{P}}$. Then, the index would 'select' some of the occurrences to provide us with our desired set $Occ_{\mathcal{T}[\ell..r]}^{\mathcal{P}}$, where either $\ell = 1$ or $r = n$.

The idea we employ is as follows. We first construct a suffix tree $ST_{\mathcal{T}}$. According to the definition of suffix tree, each leaf in $ST_{\mathcal{T}}$ is labeled by the starting location of its suffix. We do some preprocessing on $ST_{\mathcal{T}}$ as follows. We maintain a linked list of all leaves in a left-to-right order. In other words, we realize the list $LL(\mathcal{R})$ in the form of a linked list, where $\mathcal{R}$ is the root of the suffix tree. In addition to that, we set pointers $v.left$ and $v.right$ from each tree node v to its leftmost leaf v_ℓ and rightmost leaf v_r (considering the subtree rooted at v) in the linked list. It is easy to realize that, with these set of pointers at our disposal, we can indicate the set of occurrences of a pattern $\mathcal{P}$ by the two leaves $\mu_\ell^{\mathcal{P}}$ and $\mu_r^{\mathcal{P}}$ because all the leaves between and including $\mu_\ell^{\mathcal{P}}$ and $\mu_r^{\mathcal{P}}$ in $LL(\mathcal{R})$ correspond to the occurrences of $\mathcal{P}$ in $\mathcal{T}$. In what follows, we define the term ℓ_T and r_T such that $LL(\mathcal{R})[\ell_T] = \mu_\ell^{\mathcal{P}}$ and $LL(\mathcal{R})[r_T] = \mu_r^{\mathcal{P}}$, where $\mathcal{R}$ is the root of $ST_{\mathcal{T}}$. Now recall that our data structure has to be able to somehow "select" and report only those occurrences that lies in the query interval. To solve this we use a solution to the well-studied Range Minima/Maxima Query problem (Problem RMIN/MAX). In this problem, we are given an array $A[1..n]$ of numbers, which we need to preprocess to answer a batch of queries. During each query, a range is given and the goal is to find the minimum (maximum, in the case of Range Maxima Query) value in that range.

Problem RMIN/MAX has received much attention in the literature and Bender and Farach-Colton showed that we can build a data structure in $O(n)$ time using $O(n \log n)$-bit space and can answer subsequent queries in $O(1)$ time per query [3].[5] Recently, Sadakane [35] presented a succinct data structure which achieves the same time complexity using $O(n)$ bits of space. Subsequently, the space use was further reduced by Fischer and Heun [12].

[5] The same result was achieved in [14], albeit with a more complex data structure.

1: Build a suffix tree $ST_{\mathcal{T}}$ of $\mathcal{T}$. Let the root of $ST_{\mathcal{T}}$ is $\mathcal{R}$.
2: Label each leaf of $ST_{\mathcal{T}}$ by the starting location of its suffix.
3: Construct a linked list $\mathcal{L}$ realizing $LL(\mathcal{R})$. Each element in $\mathcal{L}$ is the label of the corresponding leaf in $LL(\mathcal{R})$.
4: **for** each node v in $ST_{\mathcal{T}}$ **do**
5: Store $v.left = i$ and $v.right = j$ such that $\mathcal{L}[i]$ and $\mathcal{L}[j]$ corresponds to, respectively (leftmost leaf) v_ℓ and (rightmost leaf) v_r of v.
6: **end for**
7: Preprocess $\mathcal{L}$ for both Range Minima and Range Maxima Queries.

Algorithm 1. Algorithm to build IDS_PMP/S.

Now, to complete the construction of the data structure we simply preprocess the array data structure $LL(\mathcal{R})$ for both range minima and range maxima queries. Algorithm 1 formally states the steps to build our data structure. In the rest of this paper, we refer to this data structure as IDS_PMP/S.

3.1. Analysis

Let us analyze the running time of Algorithm 1. Step 1 builds the traditional suffix tree requiring $O(n \log \sigma)$ time. Note that, for bounded alphabet the time required is reduced to $O(n)$. Step 2 can be done easily while building the suffix tree. Steps 3 and 4 can be done together in $O(n)$ by traversing $ST_{\mathcal{T}}$ using a breadth first or in order traversal. Finally, the preprocessing for range minima and range maxima queries require $O(n)$ time and space. So IDS_PMP/S can be constructed in $O(n)$ time for alphabets that are natural numbers from 1 to a polynomial in n and $O(n \log \sigma)$ time for general alphabet.

3.2. Query processing

Now we discuss the query processing. Suppose we are given a query pattern $\mathcal{P}$ along with a query interval $[\ell..r]$. We first find the locus $\mu^{\mathcal{P}}$ in $ST_{\mathcal{T}}$. Let $i = \mu^{\mathcal{P}}.left$ and $j = \mu^{\mathcal{P}}.right$. This means, we get the set $Occ_{\mathcal{T}}^{\mathcal{P}}$ in the form of $\mathcal{L}[i..j]$ spending $O(m)$ time. Now, suppose we are performing a prefix query, i.e. $\ell = 1$. So we want to compute the set $Occ_{\mathcal{T}[1..r]}^{\mathcal{P}}$. It is easy to see that

$$Occ_{\mathcal{T}[1..r]}^{\mathcal{P}} = \{\mathcal{L}[k] \mid i \leqslant k \leqslant j, \mathcal{L}[k] \leqslant r\}.$$

To compute $Occ_{\mathcal{T}[1..r]}^{\mathcal{P}}$, we apply a divide and conquer approach as follows. We perform a Range Minima Query on $\mathcal{L}$ on the interval $[i..j]$. Suppose the query returns the index k. If $\mathcal{L}[k] \leqslant r$ then $\mathcal{L}[k] \in Occ_{\mathcal{T}[1..r]}^{\mathcal{P}}$ and then we perform the range minima query on the intervals $[i..k-1]$ and $[k+1..j]$ and continue as before. If any of the queries returns k such that $L[k] > r$ we stop. It is easy to verify that this would give us the set $Occ_{\mathcal{T}[1..r]}^{\mathcal{P}}$. Note that, in this way, for each found entry in $Occ_{\mathcal{T}[1..r]}^{\mathcal{P}}$, we have at most 2 intervals to perform range minima queries further. So, in total the time spent is $O(|Occ_{\mathcal{T}[1..r]}^{\mathcal{P}}|)$.

On the other hand, for a suffix query, i.e. when $r = n$, we want to compute:

$$Occ_{\mathcal{T}[\ell..n]}^{\mathcal{P}} = \{\mathcal{L}[k] \mid i \leqslant k \leqslant j, \mathcal{L}[k] \geqslant \ell\}.$$

1: Find $\mu^{\mathcal{P}}$ in $ST_{\mathcal{T}}$.
2: Set $i = \mu^{\mathcal{P}}.left$, $j = \mu^{\mathcal{P}}.right$.
3: $Occ^{\mathcal{P}}_{\mathcal{T}[\ell..r]} = \varepsilon$
4: **if** $\ell = 1$ {This is a prefix query} **then**
5: *FindPrefixOccurrence*$(\mathcal{L}, r, i, j)$ {See Algorithm 3}
6: **else**
7: **if** $r = 1$ {This is a suffix query} **then**
8: *FindSuffixOccurrence*$(\mathcal{L}, \ell, i, j)$ {See Algorithm 4}
9: **end if**
10: **end if**

Algorithm 2. Algorithm for query processing.

1: $k = RangeMinimaQuery(\mathcal{L}, i, j)$
2: **if** $\mathcal{L}[k] < r$ **then**
3: Set $Occ^{\mathcal{P}}_{\mathcal{T}[\ell..r]} = Occ^{\mathcal{P}}_{\mathcal{T}[\ell..r]} \cup \mathcal{L}[k]$
4: *FindPrefixOccurrence*$(\mathcal{L}, r, i, k-1)$
5: *FindPrefixOccurrence*$(\mathcal{L}, r, k+1, j)$
6: **end if**

Algorithm 3. Procedure *FindPrefixOccurrence*$(\mathcal{L}, r, i, j)$.

1: $k = RangeMaximaQuery(\mathcal{L}, i, j)$
2: **if** $\mathcal{L}[k] > \ell$ **then**
3: Set $Occ^{\mathcal{P}}_{\mathcal{T}[\ell..r]} = Occ^{\mathcal{P}}_{\mathcal{T}[\ell..r]} \cup \mathcal{L}[k]$
4: *FindSuffixOccurrence*$(\mathcal{L}, \ell, i, k-1)$
5: *FindSuffixOccurrence*$(\mathcal{L}, \ell, k+1, j)$
6: **end if**

Algorithm 4. Procedure *FindSuffixOccurrence*$(\mathcal{L}, \ell, i, j)$.

So, in this case, in the above procedure, we just need to perform a Range Maxima (instead of Minima) Query and instead of checking whether $\mathcal{L}[k] \leqslant r$, we need to check whether $\mathcal{L}[k] \geqslant \ell$. The query steps are formally stated in Algorithms 2, 3 and 4. In light of the above discussion, it is straightforward to see that the total query time is $O(m + |Occ^{\mathcal{P}}_{\mathcal{T}[\ell..r]}|)$. Since the size of the pattern is m and the size of the output is $|Occ^{\mathcal{P}}_{\mathcal{T}[\ell..r]}|$, the query time achieved is optimal.

We note that the divide and conquer technique that we have used to devise IDS_PMP/S has been used in [30] to efficiently solve a different problem namely, the *document listing problem*. In document listing problem, the goal is to prepare an index for a library of documents to support a special type of queries, where instead of all the occurrences of a given pattern, only the documents where the pattern is present are sought for. We believe that this technique could be further explored to solve different other interesting problems.

One final remark is that, we can use the suffix array instead of suffix tree as well with some standard modifications in the algorithms. This might be useful because, practically, suffix arrays use less space than suffix trees. We can achieve space efficiency further at the cost of slight increase in the query time. For example we can use any FM-index variants or compressed suffix arrays (see [31]) to gain more space efficiency. The resulting query time will however suffer a bit and would not remain optimal.

4. Extensions

In this section, we discuss two interesting algorithmic problems that can be solved using the data structure IDS_PMP/S described in the previous section. The first problem is to find a given pattern outside a given range.

Problem "PMCQI" *(Pattern matching in a complementary query interval).* Suppose we are given a text $\mathcal{T}$ of length n. Preprocess $\mathcal{T}$ to answer following form of queries.

Query: Given a pattern $\mathcal{P}$ and an interval $[\ell..r]$, with $1 \leqslant \ell \leqslant r \leqslant n$, construct the set

$$Occ^{\mathcal{P}}_{\mathcal{T}[\overline{\ell..r}]} = \left\{ i \mid i \in Occ^{\mathcal{P}}_{\mathcal{T}} \text{ and } i \notin [\ell..r] \right\}.$$

Problem PMCQI, can be seen as a 'Complementary' version of the problem handled in [28]. Indeed, the difference between the two problems is that, in the latter, the given query interval is 'interesting', whereas in the former, it is the complement of that interval, which is of interest. Problem PMCQI has applications in cases where a particular range is not interesting anymore, for example, due to a previous search. Using the solution of [28], we can easily solve Problem PMCQI, although sub-optimally, by transforming the 'complementary' query interval $[\ell..r]$, given as the input, to two corresponding intervals $[1..\ell-1]$ and $[r+1..n]$. But we can use the IDS_PMP/S to get optimal query time, since the two resulting intervals gives, respectively, prefix and suffix of the text.

Another interesting problem, where IDS_PMP/S can be efficiently used, is the problem of finding all the 'borders' of a string. A substring $u \neq X$ of X is said to be a border of X if, and only if, it is both a prefix and a suffix of X. Computation of all the borders is used in various related algorithms, e.g., algorithms to compute the covers[6] of a string. We can use IDS_PMP/S to compute all the borders of a given string X as follows. The idea is to build IDS_PMP/S for X and then 'feed' itself into IDS_PMP/S. Now, consider a prefix $X[1..i]$, $1 \leqslant i \leqslant n$. From the description of the query processing in Section 3.2, it is easy to see that using IDS_PMP/S we can easily find the largest k such that $X[1..i]$ occurs at position k of X. Now, if $|X| - k + 1 = i$, then $X[1..i]$ is a border. Thus, given IDS_PMP/S, we can find all the borders of X in $O(|X|)$ time. Recall that the construction of IDS_PMP/S requires $O(|X|)$ time as well. We remark that the computation of all the borders can be done easily using the classic Morris–Pratt algorithm [21]. However, with IDS_PMP/S at our disposal, we have the added functionality to answer following interesting online queries regarding borders and periods of a string. It is easy to see that, given a prefix $X[1..\ell]$, we can answer whether $X[1..\ell]$ is a border or not in $O(\ell)$ time. Similarly, the query, whether there exists a border of length ℓ, can be answered analogously. Equivalently, we can answer the query, whether $|X| - \ell$ is a period of X.[7]

Furthermore, as is mentioned in the introduction, IDS_PMP/S can be used to compute the suffix–prefix match efficiently. This can be done as follows. We create the IDS_PMP/S for S_1. Now, we traverse the data structure spelling S_2. As soon as we get a character mismatch we stop. Suppose we had to stop spelling $S_2[1..k]$. Now we find the rightmost occurrence of $S_2[1..k]$ in S_1 and check whether this occurrence is a suffix of S_1. This can be

[6] A substring $u \neq X$ of X is said to be a cover of X if, and only if, every position of X lies within an occurrence of u in X.

[7] $|X| - \ell$ is a period of X if, and only, if $X[1..\ell]$ is a border.

done easily in constant time using IDS_PMP/S. If the answer is positive, then we are done. Otherwise we perform the same check for $S_2[1..k-1]$ and so on. Therefore, it is easy to verify that, given IDS_PMP/S for S_1, we can compute suffix–prefix match of (S_1, S_2) in $O(|S_2|)$ time. We note that using a variant of the well-known KMP algorithm [24], the suffix–prefix matching problem can be solved in $O(|S_2| + |S_1|)$ time which is the same as the algorithm above. However, our algorithm would be suitable in the context of indexing when we are looking for suffix–prefix matches of several strings against a fixed string S_1.

5. Conclusion

In this paper, we have studied Problem PMP/S, a restricted version of the position restricted pattern matching problem (Problem PRPM) introduced and studied in [28]. In Problem PRPM, the query provides an interval $[\ell..r]$, $1 \leqslant \ell \leqslant r \leqslant n$, along with the pattern $\mathcal{P}$ and the occurrences of $\mathcal{P}$ in $\mathcal{T}[\ell..r]$ are sought for. In Problem PMP/S, on the other hand, we are interested in those occurrences of the pattern that lies in a suffix or in a prefix of the given text. In other words, in our case, the query interval $[\ell..r]$ is of special form: either $\ell = 1$, i.e. prefix search, or $r = n$, i.e. suffix search. We have presented an efficient data structure, IDS_PMP/S, which is an extension of the classic suffix tree data structure. The time and space complexity of IDS_PMP/S is dominated by that of the suffix tree and hence is $O(n)$ for bounded alphabet and $O(n \log \Sigma)$ for the general case. The query time we achieve is $O(m + |Occ^{\mathcal{P}}_{\mathcal{T}[\ell..r]}|)$ time per query, which is optimal. Notably, the (best) algorithm in [28], if applied to Problem PMP/S, gives sub-optimal query time and the corresponding data structure also requires more time and space. We also note that the method we have presented cannot be used to get better results for the general case handled in [28].

Finally, we have shown that our data structure IDS_PMP/S can be used to solve optimally the 'complementary' version of Problem PRPM. Also, IDS_PMP/S can be used to compute all the borders of a given string and to answer some related queries and to compute the prefix–suffix match between two strings. One interesting feature is that, with IDS_PMP/S, we can answer 'normal' pattern queries,[8] especial pattern queries (Problem PMP/S, PMCQI), as well as, queries related to borders and periods. This, we believe, makes our data structure a very strong tool to be used in different pattern matching and related applications.

Acknowledgement

The authors would like to express their gratitude to the anonymous reviewers for their helpful suggestions.

References

[1] A. Amir, D. Keselman, G.M. Landau, M. Lewenstein, N. Lewenstein, M. Rodeh, Text indexing and dictionary matching with one error, J. Algorithms 37 (2) (2000) 309–325.

[2] A. Apostolico, The myriad virtues of subword trees, in: Combinatorial Algorithms on Words, in: NATO ISI, Springer-Verlag, 1985, pp. 85–96.

[3] M.A. Bender, M. Farach-Colton, The LCA problem revisited, in: G.H. Gonnet, D. Panario, A. Viola (Eds.), Latin American Theoretical INformatics (LATIN), in: Lecture Notes in Computer Science, vol. 1776, Springer, 2000, pp. 88–94.

[4] A. Blum, T. Jiang, M. Li, J. Tromp, M. Yannakakis, Linear approximation of shortest superstrings, in: STOC, ACM, 1991, pp. 328–336.

[5] H.-L. Chan, W.-K. Hon, T.W. Lam, Compressed index for a dynamic collection of texts, in: S.C. Sahinalp, S. Muthukrishnan, U. Dogrusöz (Eds.), CPM, in: Lecture Notes in Computer Science, vol. 3109, Springer, 2004, pp. 445–456.

[6] R. Cole, L.-A. Gottlieb, M. Lewenstein, Dictionary matching and indexing with errors and don't cares, in: L. Babai (Ed.), STOC, ACM, 2004, pp. 91–100.

[7] M. Crochemore, C.S. Iliopoulos, M. Kubica, M.S. Rahman, T. Walen, Improved algorithms for the range next value problem and applications, in: S. Albers, P. Weil, C. Rochange (Eds.), STACS, in: Dagstuhl Seminar Proceedings, vol. 08001, Internationales Begegnungs- und Forschungszentrum fuer Informatik (IBFI), Schloss Dagstuhl, Germany, 2008, pp. 205–216.

[8] M. Crochemore, C.S. Iliopoulos, M.S. Rahman, Finding patterns in given intervals, in: L. Kucera, A. Kucera (Eds.), MFCS, in: Lecture Notes in Computer Science, vol. 4708, Springer, 2007, pp. 645–656.

[9] M. Crochemore, W. Rytter, Jewels of Stringology, World Scientific, 2002.

[10] M. Farach, Optimal suffix tree construction with large alphabets, in: FOCS 1997, pp. 137–143.

[11] P. Ferragina, R. Grossi, Fast incremental text editing, in: SODA, 1995, pp. 531–540.

[12] J. Fischer, V. Heun, A new succinct representation of RMQ-information and improvements in the enhanced suffix array, in: B. Chen, M. Paterson, G. Zhang (Eds.), ESCAPE, in: Lecture Notes in Computer Science, vol. 4614, Springer, 2007, pp. 459–470.

[13] T. Flouri, C.S. Iliopoulos, M.S. Rahman, L. Vagner, M. Vorácek, Indexing factors in DNA/RNA sequences, in: BIRD08-ALBIO08, 2008, in press.

[14] H. Gabow, J. Bentley, R. Tarjan, Scaling and related techniques for geometry problems, in: Symposium on the Theory of Computing (STOC), 1984, pp. 135–143.

[15] M. Gu, M. Farach, R. Beigel, An efficient algorithm for dynamic text indexing, in: SODA, 1994, pp. 697–704.

[16] D. Gusfield, Algorithms on Strings, Trees, and Sequences—Computer Science and Computational Biology, Cambridge University Press, 1997.

[17] D. Gusfield, G.M. Landau, B. Schieber, An efficient algorithm for the all pairs suffix–prefix problem, Inform. Process. Lett. 41 (4) (1992) 181–185.

[18] C.S. Iliopoulos, M.S. Rahman, Indexing factors with gaps, Algorithmica, doi:10.1007/s00453-007-9141-3, in press.

[19] C.S. Iliopoulos, M.S. Rahman, Faster index for property matching, Inform. Process. Lett. 105 (6) (2008) 218–223.

[20] C.S. Iliopoulos, M.S. Rahman, Indexing circular patterns, in: S.-I. Nakano, M.S. Rahman (Eds.), WALCOM, in: Lecture Notes in Computer Science, vol. 4921, Springer, 2008, pp. 46–57.

[21] J.M. Jr., V. Pratt, A linear pattern matching algorithm, University of California Technical report, 40, 1970.

[22] J.D. Kececioglu, E.W. Myers, Combinatorial algorithms for DNA sequence assembly, Algorithmica 13 (1–2) (1995) 7–51.

[23] Z.M. Kedem, G.M. Landau, K.V. Palem, Parallel suffix–prefix-matching algorithm and applications, SIAM J. Comput. 25 (5) (1996) 998–1023.

[24] D.E. Knuth, J.H.M. Jr., V.R. Pratt, Fast pattern matching in strings, SIAM J. Comput. 6 (2) (1977) 323–350.

[25] A. Lesk, Computational Molecular Biology: Sources and Methods for Sequence Analysis, Oxford University Press, Inc., New York, NY, USA, 1988.

[26] M.G. Maaß, J. Nowak, Text indexing with errors, in: A. Apostolico, M. Crochemore, K. Park (Eds.), CPM, in: Lecture Notes in Computer Science, vol. 3537, Springer, 2005, pp. 21–32.

[27] V. Mäkinen, G. Navarro, Dynamic entropy-compressed sequences and full-text indexes, in: M. Lewenstein, G. Valiente (Eds.), CPM, in: Lecture Notes in Computer Science, vol. 4009, Springer, 2006, pp. 306–317.

[8] Recall that the suffix tree is still there.

[28] V. Mäkinen, G. Navarro, Position-restricted substring searching, in: J.R. Correa, A. Hevia, M.A. Kiwi (Eds.), LATIN, in: Lecture Notes in Computer Science, vol. 3887, Springer, 2006, pp. 703–714.
[29] E.M. McCreight, A space-economical suffix tree construction algorithm, J. ACM 23 (2) (1976) 262–272.
[30] S. Muthukrishnan, Efficient algorithms for document retrieval problems, in: SODA, 2002, pp. 657–666.
[31] G. Navarro, V. Mäkinen, Compressed full-text indexes, ACM Comput. Surv. 39 (1) (2007).
[32] G. Navarro, E. Sutinen, J. Tanninen, J. Tarhio, Indexing text with approximate q-grams, in: R. Giancarlo, D. Sankoff (Eds.), CPM, in: Lecture Notes in Computer Science, vol. 1848, Springer, 2000, pp. 350–363.
[33] M.S. Rahman, C.S. Iliopoulos, Indexing factors with gaps, in: J. van Leeuwen, G.F. Italiano, W. van der Hoek, C. Meinel, H. Sack, F. Plasil, M. Bieliková (Eds.), SOFSEM, in: Lecture Notes in Computer Science, vol. 4362, Springer, 2007, pp. 465–474.
[34] M.S. Rahman, C.S. Iliopoulos, I. Lee, M. Mohamed, W.F. Smyth, Finding patterns with variable length gaps or don't cares, in: D.Z. Chen, D.T. Lee (Eds.), COCOON, in: Lecture Notes in Computer Science, vol. 4112, Springer, 2006, pp. 146–155.
[35] K. Sadakane, Succinct data structures for flexible text retrieval systems, J. Discrete Algorithms 5 (1) (2007) 12–22.
[36] J. Tarhio, E. Ukkonen, A greedy approximation algorithm for constructing shortest common superstrings, Theoret. Comput. Sci. 57 (1988) 131–145.
[37] E. Ukkonen, A linear-time algorithm for finding approximate shortest common superstrings, Algorithmica 5 (3) (1990) 313–323.
[38] E. Ukkonen, On-line construction of suffix trees, Algorithmica 14 (3) (1995) 249–260.